Clean-Aware CP-Guided Large Neighborhood Search with Event-Level Validation for Semiconductor Cluster Tool Scheduling

Sen Li[1,2], Te Xu[3,4], Defeng Sun*[3,4], and Lixin Tang[1]

[1]*National Frontiers Science Center for Industrial Intelligence and Systems Optimization, Northeastern University, Shenyang 110819, China*

[2]*Key Laboratory of Data Analytics and Optimization for Smart Industry (Northeastern University), Ministry of Education, China*

[3]*Liaoning Key Laboratory of Manufacturing System and Logistics Optimization, Shenyang 110819, China*

[4]*Liaoning Engineering Laboratory of Data Analytics and Optimization for Smart Industry, Shenyang 110819, China*

Abstract

We study a semiconductor cluster tool scheduling problem with just-in-time time-lag constraints, blocking, reentrant wafer routes, state-dependent chamber cleaning, robot transfer rules, valve coordination, and an executable MoveList output requirement. The combination of flexible process-module assignment, cleaning state, blocking occupancy, and event-level transfer feasibility makes compact machine schedules insufficient for evaluating solution quality. We develop a clean-aware constraint-programming-guided large neighborhood search method that couples constructive initial scheduling, data-guided destroy-and-repair search, CP-based neighborhood repair, event-level decoding, and validation. The method keeps cleaning state across construction, repair, replay, and validation, with sequence-dependent cleaning modeled in CP repair and idle-triggered cleaning activated after realized gaps are known. The validator checks the resulting MoveList against physical transition, robot-domain, overlap, handoff, and realized JIT conditions before a schedule is accepted. Computational results on eight NAURA Cup benchmark instances show validator-feasible schedules for all tested cases. The method improves the reported objective values of Liu et al. [**?**] on six instances, while the two remaining instances show that performance is sensitive to neighborhood selection.

Keywords: Semiconductor manufacturing, cluster tool scheduling, constraint programming, large neighborhood search, just-in-time time lag, event-level validation.

*Corresponding author: sundefeng@ise.neu.edu.cn

1 Introduction

Semiconductor cluster tools are highly integrated equipment cells in which load ports, transfer robots, load locks, and process modules must be coordinated at the level of individual wafer movements. Each wafer is released from a cassette, aligned, processed through a route-dependent sequence of process-module operations, cooled, and returned to its slot. The resulting schedule must determine module assignment, local module sequencing, robot transfers, load-lock operations, blocking occupancy, valve coordination, cleaning, and timing relations.

The benchmark problem considered in this study is difficult for three reasons. First, each wafer route may contain flexible process-module choices and reentrant visits, such that local assignment decisions interact across the route. Second, timing feasibility is not captured by processing intervals alone. A wafer may block a module after processing, and the subsequent handoff must satisfy just-in-time dwell and move limits. Third, the required output is an executable MoveList rather than only a machine-level schedule. A compact schedule can therefore be accepted only after it has been decoded into event-level actions and checked against detailed equipment rules.

Cluster tool scheduling has been studied through timed-Petri-net models, mathematical programming, process-module sharing, transient and cyclic scheduling, residency constraints, and robot idle-time control [**?** **?** **?** **?** **?** **?**]. These studies provide important foundations, but the benchmark setting considered here requires a broader combination of flexible process-module assignment, reentrant routes, state-dependent cleaning, blocking, JIT transfer timing, and executable movement validation. The main computational question is therefore not only how to optimize a compact schedule, but also how to ensure that the optimized schedule can be realized as equipment-level actions.

We develop a clean-aware CP-guided large neighborhood search method for this setting. The method builds an initial compact schedule, repeatedly relaxes selected neighborhoods, repairs each neighborhood by CP-SAT, decodes accepted skeletons into event-level actions, and validates the realized schedule before it is reported. Cleaning is treated as a state-dependent constraint rather than as a fixed setup time. Family-switch and count-triggered cleanings are propagated through machine arcs in the compact model, while idle-triggered cleanings are reconstructed during replay and validation after realized idle gaps are known.

This study contributes to the benchmark problem in three ways. First, it gives a layered formulation of the equipment rules, distinguishing constraints enforced in the compact optimization model from those enforced by replay, decoding, and validation. Second, it presents a CP-based neighborhood repair model with explicit assignment, timing, blocking, sequencing, and cleaning-state variables. Third, it evaluates the method on the eight NAURA Cup instances, including sensitivity, short-horizon, and ablation analyses that clarify the roles of the initial construction, CP repair, and neighborhood selection.

The remainder of the paper is organized as follows. Section **??** defines the problem setting, notation, and constraint layers. Section **??** describes the solution method. Section **??** presents the CP repair model. Section **??** explains event-level decoding and validation. Section **??** reports computational results. Section **??** describes implementation details, and Section **??** concludes.

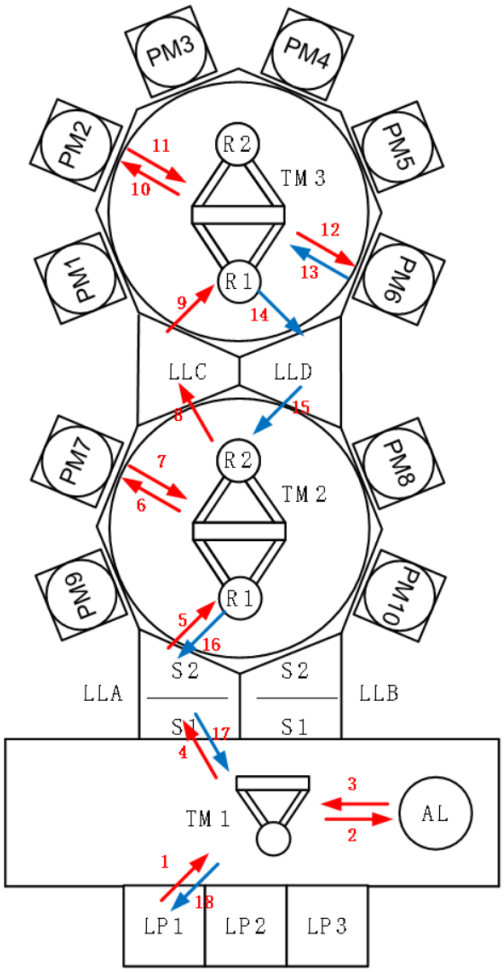

Figure 1: Dual-transfer-chamber cluster tool used in the benchmark setting. TM1 serves the atmospheric side, while TM2 and TM3 serve the vacuum region through load locks LLA–LLD.

2 Problem Description and Modeling Layers

2.1 Equipment and wafer routes

We consider a semiconductor cluster tool scheduling problem in which each wafer is a job. A job starts in a load port cassette, is calibrated at an aligner, enters the vacuum region through a load lock, visits a route-dependent sequence of process modules, and returns to a load lock for cooling before returning to its original slot. The physical equipment has both atmospheric and vacuum handling regions. The atmospheric side contains the load ports, aligner, and front transfer module. The vacuum side contains process modules and load locks connected by transfer modules.

Figure **??** shows the equipment structure. The scheduler must assign each process step to an eligible process module, choose the local processing order on each module, and ensure that the event-level wafer movement

is physically possible. The route of a wafer may include repeated visits to similar modules, which creates reentrant flow. Wafers in the same cassette and route may also be subject to release, output, and no-overtake rules.

Blocking denotes the occupancy of a module after processing finishes. If a wafer cannot be immediately transferred out of a process module, it remains in that module and blocks the next wafer. The schedule therefore distinguishes the processing interval of an operation from its blocking occupancy interval. The JIT time-lag limits are imposed on realized event times. The benchmark uses a 15-second upper bound for dwell after process completion and a 30-second upper bound for the move from one operation to the next after mandatory transition time has been subtracted.

2.2 Decisions and objective

Let $\mathcal{J}$ be the set of wafer jobs. For each job $j \in \mathcal{J}$, let

$$\mathcal{O}_j = (o_{j1}, o_{j2}, \ldots, o_{jn_j}) \tag{1}$$

denote its ordered list of modeled operations. The modeled operations include alignment, process-module operations, transfer-buffer operations where needed by the abstraction, and final cooling. For each operation o, let $\mathcal{M}_o$ be its eligible machine set and let p_o be its duration.

The scheduler determines the selected machine for each flexible operation, the local sequence on each machine, the processing start and completion times, the release time of each blocking occupancy, the cleaning placement induced by machine state, and an executable event-level MoveList. The reported objective is the realized makespan after event-level decoding and validation. We write the compact objective as

$$\min C_{\max}, \tag{2}$$

where $C_{\max}$ is linked to the completion of the final modeled operation of each job. The implementation records both compact skeleton timing and realized event timing. The realized value is used for the reported objective when the decoder produces additional unavoidable event-level delays.

For operation o, let

$$P_o = [S_o, C_o), \qquad B_o = [S_o, D_o) \tag{3}$$

be its processing interval and blocking occupancy interval, where S_o is the compact start time, C_o the processing completion time, and $D_o \geq C_o$ the release time of the machine occupancy. The difference between C_o and D_o allows the compact schedule to represent blocking.

Table 1: Main notation.

Symbol	Domain or type	Meaning	Main use
$\mathcal{J}$	finite set	wafer jobs	problem definition
j	$j \in \mathcal{J}$	wafer job index	route and objective definitions
$\mathcal{O}_j$	ordered list	modeled operations of job j	route precedence
o, o'	operation indices	modeled operations, including alignment, processing, and cooling operations	CP repair and validation
$\mathcal{M}_o$	finite set	eligible machines for operation o	flexible machine assignment
m	module index	processing module, load lock, aligner, or related modeled resource	machine sequencing
p_o	nonnegative integer	processing, alignment, or cooling duration of operation o	timing constraints
S_o	integer time	start time of operation o in the compact schedule	CP repair
C_o	integer time	processing completion time of operation o in the compact schedule	CP repair
D_o	integer time	release time of the blocking occupancy of operation o	blocking and precedence
P_o	interval	processing interval $[S_o, C_o)$	conceptual model
B_o	interval	blocking occupancy interval $[S_o, D_o)$	machine occupancy
Ω	subset of operations	relaxed neighborhood selected by LNS	CP repair
Ω_m	subset of operations	operations in Ω that may use machine m	machine circuit constraints
h_m	anchor node	frozen head anchor before the repaired window on machine m	boundary consistency
t_m	anchor node	frozen tail anchor after the repaired window on machine m	boundary consistency

Continued on next page

Symbol	Domain or type	Meaning	Main use
$\bar{D}_{h_m}$	integer time	release time of the head anchor on machine m	head-link constraints
$\bar{S}_{t_m}$	integer time	start time of the tail anchor on machine m	tail-link constraints
H_m	positive integer	count-trigger cleaning threshold on machine m	cleaning state propagation
Θ_m	nonnegative integer	idle-trigger cleaning threshold on machine m	replay and validation
q_o^m	integer in $\{1, \ldots, H_m\}$	number of wafers processed on machine m since the last cleaning after operation o	count-trigger cleaning
$x_{o,m}$	binary	one if operation o is assigned to machine m	machine assignment
$a_{u,v,m}$	binary	one if node v immediately follows node u on machine m	local machine circuit
$r^{\mathrm{sw}}_{u,v,m}$	binary	family-switch cleaning trigger on boundary (u, v, m)	cleaning setup duration
$r^{\mathrm{cnt}}_{u,v,m}$	binary	count-trigger cleaning trigger on boundary (u, v, m)	cleaning setup duration
$r^{\mathrm{idl}}_{u,v,m}$	binary	idle-trigger cleaning indicator on boundary (u, v, m) after realized gaps are known	replay and validation
$\ell^{\mathrm{man}}_{u,v,m}$	nonnegative integer	mandatory cleaning duration caused by family switch or count trigger	temporal separation
$g_{u,v,m}$	integer time	realized idle gap between adjacent operations on machine m	idle cleaning
$C_{\max}$	integer time	compact makespan or reported realized makespan, depending on context	objective
C_o^{real}	real time	realized completion time of the process-like action for operation o	event validation
D_o^{real}	real time	realized completion time of the first leave-side handoff after operation o	JIT dwell check

Continued on next page

Symbol	Domain or type	Meaning	Main use
$A_{o'}^{\mathrm{real}}$	real time	realized arrival or handoff completion time at the next operation o'	JIT move check
$\tau_o^{\mathrm{need,leave}}$	nonnegative real time	mandatory transition time required to leave the current node	JIT dwell check
$\tau_{o,o'}^{\mathrm{need,total}}$	nonnegative real time	mandatory transition time required to move from o to o'	JIT move check
W_k	nonnegative weight	adaptive weight of destroy operator k	data-guided LNS
R_k	real reward	realized search reward assigned to destroy operator k	adaptive update

2.3 State-dependent cleaning

The method is called *clean-aware* because the search keeps the process-module cleaning state as part of schedule construction, local repair, replay, and validation. This is stronger than adding a fixed setup time between process steps. The cleaning decision depends on the previous route family on a module, the number of wafers processed since the last cleaning, and the realized idle gap before the next operation.

Three cleaning mechanisms are distinguished. A family-switch cleaning is triggered when consecutive operations on the same process module belong to different route families. A count-triggered cleaning is triggered when the post-cleaning wafer count reaches the module threshold. These two mechanisms depend on the local machine sequence and can be represented inside CP repair through arc-dependent durations and counter propagation. An idle-triggered cleaning depends on the realized gap between module occupancies. Since the realized gap may change after decoding and replay, this cleaning is applied after the gap pattern is known.

This layered treatment is intentional. Modeling every event-level cleaning alternative inside each CP neighborhood would make the local model much larger. The implemented method instead lets CP optimize a compact skeleton with mandatory sequence-dependent cleanings and then uses replay and validation to activate idle-triggered cleanings and reject schedules whose realized event trace violates equipment rules.

2.4 Layered constraint enforcement

Table **??** summarizes how the main equipment and scheduling rules are enforced. The compact CP model is a neighborhood optimizer for assignment, timing, sequencing, blocking, and mandatory cleaning. Replay, decoding, and validation complete the feasibility check at the event level.

Table 2: Constraint coverage across optimization, replay, decoding, and validation layers.

Constraint group	Operational rule	Optimization or replay layer	Decoder or validator layer
Route precedence	A wafer follows its modeled route order.	CP enforces predecessor release before successor start.	Realized process-like actions are checked along each job arc.
Flexible PM assignment	Each process step must use an eligible PM.	Assignment variables choose one candidate machine.	Invalid machine choices are absent from decoded schedules.
Blocking	A wafer can occupy a PM after processing until the next transfer is feasible.	Release variables satisfy $D_o \geq C_o$ and are sequenced on machines.	Realized starts earlier than the skeleton are rejected.
No-overtake	Wafers with the same cassette, route, step, and PM preserve wafer order.	Conditional order constraints are added inside repaired neighborhoods.	Buckets by cassette, route, step, and machine are checked after decoding.
Output order	Wafers from the same cassette and route return in prescribed order.	Final-operation order constraints are included when relevant operations are repaired.	Final completion order is checked for each cassette-route group.
Sequential release	Later cassettes cannot be released before earlier cassettes under the sequential policy.	First-operation start order is constrained in CP.	The release policy is checked on first modeled operations.
Family-switch cleaning	A PM may require cleaning when the route family changes.	Arc-dependent mandatory duration is inserted in CP and replay.	Cleaning events are recomputed in machine-window validation.
Count-trigger cleaning	A PM may require cleaning after a threshold number of processed wafers.	Post-clean counters propagate along machine arcs.	Machine-window validation recomputes threshold-triggered cleanings.
Idle-trigger cleaning	A PM may require cleaning after a long idle period.	Applied during replay once idle gaps are known.	Machine-window validation recomputes idle-triggered cleanings.
JIT dwell and move limits	Waiting after process completion and before the next operation is bounded.	CP uses compact dwell and move guidance.	Realized dwell and move excesses are checked after mandatory transition times are subtracted.
Physical transitions	Robots must move wafers only through feasible source and destination pairs.	Not fully represented in the compact CP skeleton.	Robot domains, handoff continuity, and impossible transitions are checked on the MoveList.
MoveList executability	The final output must be an executable sequence of actions.	Accepted schedules must pass post-repair validation.	The decoder builds actions and the validator rejects infeasible event traces.

3 Solution Method

The method combines a constructive initial schedule, large neighborhood search, CP-based repair, and event-level validation. Algorithm **??** gives the workflow. The initial schedule provides the starting point for the search loop. Each iteration selects a destroy operator, builds a relaxed neighborhood, repairs it with CP, decodes the result, and accepts only schedules that pass the implemented validator and satisfy the acceptance criterion.

3.1 Initial solution construction

The initial construction builds a schedule before the LNS loop begins. It schedules each wafer as a no-wait route-level block in the compact model. Once the first operation of a wafer is assigned a start time, the remaining modeled operations are placed along the route using the corresponding processing durations. This structure avoids large waiting gaps between consecutive operations and gives CP repair a stable initial sequence.

The construction has two stages. The ordering stage determines the wafer order, and the assignment stage selects a route-level machine pattern for each wafer. A simple route-SPT rule orders route groups by total modeled processing time and then follows wafer order within each group. The revised rule uses an interleaved wafer order, where wafers from different routes, cassettes, or blocks can be mixed according to the instance structure.

Algorithm 1: Clean-aware CP-guided LNS

Input: Instance data, global time limit T, CP repair limit T_{cp}
Output: Best validated event-level schedule $s^\star$

$s \leftarrow$ ConstructInitialSchedule()
$s^\star \leftarrow s$
Initialize W from instance features
while *elapsed time* $< T$ **do**
  $k \leftarrow$ SelectOperator(W)
  $\Omega \leftarrow$ Destroy(s, k)
  $(\bar{s}, \Sigma^\partial) \leftarrow$ FreezeOutsideNeighborhood(s, Ω)
  $\hat{s} \leftarrow$ CPRepair($\bar{s}, \Omega, \Sigma^\partial, T_{\text{cp}}$)
  if $\hat{s}$ *is feasible* **then**
    $(\tilde{s}, \mathcal{V}) \leftarrow$ DecodeValidate($\hat{s}$)
    if $\mathcal{V} = \emptyset$ *and* Accept($\tilde{s}, s$) **then**
      $s \leftarrow \tilde{s}$
      if $C_{\max}(s) < C_{\max}(s^\star)$ **then**
        $s^\star \leftarrow s$
    UpdateWeights($W, k, s, \tilde{s}$)
  else
    PenalizeOperator(W, k)
return EventSchedule($s^\star$)

Given this order, machine patterns are evaluated at the route level rather than operation by operation. The scoring function considers projected completion time, cleaning burden, family switches, count-trigger risk, idle cleaning, and machine-load balance. After a pattern is committed, machine availability, last route family, and post-cleaning counters are updated. The construction therefore returns both a schedule and boundary-state information for later repair.

3.2 Destroy operators and data-guided adaptation

The search uses six destroy operators. Critical-window neighborhoods focus on operations near the current tail-critical region. Flexible-reassignment neighborhoods relax alternative process-module choices. Family-boundary neighborhoods target cleaning-sensitive boundaries. Release-wave neighborhoods perturb groups of early route starts. Time-window neighborhoods relax congested intervals. Bottleneck-chain neighborhoods follow machine segments that appear to constrain the current makespan.

The data-guided component performs online adaptation of destroy-operator weights. The initial weights depend on simple instance features, including release policy, number of route families, and whether reentrant routes are present. During the search, the selected operator receives a reward or penalty based on the change in the current energy score, which includes realized objective value, cleaning duration, bottleneck tail, and suffix shift penalties. The weight is updated through an exponential smoothing rule,

$$W_k \leftarrow (1 - \rho)W_k + \rho(1 + \bar{R}_k), \tag{4}$$

Table 3: Main components of the CP-guided LNS method.

Component	Operation	Role
Initial construction	Build a no-wait compact schedule with route-group ordering	Provides a feasible backbone and initial cleaning state
Decode and validate	Expand the compact schedule into events and check feasibility	Ensures that accepted schedules pass the implemented event-level feasibility checks
Operator selection	Choose a destroy operator using adaptive weights	Biases the search toward neighborhoods that work on the current instance
Destroy	Relax selected operations and machine windows	Performs macro-level structural exploration
CP repair	Reassign, resequence, and reschedule the relaxed part	Performs exact or time-limited local optimization inside the neighborhood
Acceptance and update	Accept a candidate and update operator weights	Controls intensification, diversification, and online adaptation

where $\bar{R}_k$ is a scaled reward for operator k. This adaptation uses data collected during the current run and does not rely on an offline learning policy.

After CP repair, the candidate is decoded and validated. Candidates failing validation are rejected. Feasible candidates are compared with the incumbent using an energy score that includes realized makespan, compact makespan, cleaning duration, bottleneck tail, and suffix movement. The acceptance rule allows strict improvements, plateau moves that change machine arcs without worsening cleaning, and simulated-annealing moves when the energy increase is small relative to the current temperature.

4 CP-Based Neighborhood Repair

The repair problem is solved in a compact CP-SAT neighborhood model after a destroy operator selects a set of relaxed operations. The model optimizes a machine-level skeleton with assignment, timing, sequencing, blocking, and sequence-dependent cleaning separations. Event-level physical feasibility is checked after decoding, as described in Section **??**.

4.1 Neighborhood, anchors, and variables

Let Ω be the set of relaxed operations. Operations outside Ω remain fixed unless an elastic suffix shift is explicitly allowed. For each affected machine m, the fixed sequence outside the repaired window defines a head anchor h_m and a tail anchor t_m. The head anchor provides a boundary release time $\bar{D}_{h_m}$, a last-family state, and a post-cleaning count. The tail anchor provides a boundary start time $\bar{S}_{t_m}$ and a family label. These anchors make the local repair compatible with the frozen part of the incumbent schedule.

Table **??** lists the main CP variables. These variables express the local assignment, sequence, timing, blocking, and cleaning-state decisions.

Table 4: Main CP repair variables.

Variable	Domain	Meaning
S_o	integer time	Start time of relaxed operation o.
C_o	integer time	Processing completion time of relaxed operation o.
D_o	integer time	Blocking release time of relaxed operation o.
$x_{o,m}$	binary	One if operation o is assigned to machine m.
$a_{u,v,m}$	binary	One if node v immediately follows node u on machine m.
q_o^m	$\{1, \ldots, H_m\}$	Count since the last cleaning after operation o on machine m.
b_o^m	binary	One if $q_o^m = H_m$.
Δ^{tail}	nonnegative integer	Elastic shift applied to frozen suffix operations when enabled.
$C_{\max}$	integer time	Compact makespan used in the local objective.

4.2 Assignment, timing, and compact JIT guidance

Each relaxed operation chooses exactly one eligible candidate machine:

$$\sum_{m \in \mathcal{M}_o} x_{o,m} = 1, \qquad o \in \Omega. \tag{5}$$

The timing variables satisfy

$$C_o = S_o + p_o, \qquad o \in \Omega, \tag{6}$$

$$D_o \geq C_o, \qquad o \in \Omega. \tag{7}$$

The model also keeps a compact dwell bound inside CP repair:

$$D_o - C_o \leq 15, \qquad o \in \Omega. \tag{8}$$

For a technological arc (o, o'), if both operations are relaxed, the compact precedence and move guidance are

$$S_{o'} \geq D_o, \tag{9}$$

$$S_{o'} - D_o \leq 30. \tag{10}$$

If one endpoint is outside the neighborhood, the fixed skeleton time of that endpoint is used, with an optional tail shift when suffix elasticity is enabled.

Equations (**??**) and (**??**) are guidance constraints for the compact skeleton. They do not replace the event-level JIT check. The realized dwell and move limits are evaluated after the decoder has inserted mandatory pick, place, transfer, pump, vent, prepare, and complete actions.

4.3 Machine circuits and cleaning state

For each affected machine m, CP builds a local circuit over the operations that may use m, plus a depot node representing the boundary of the repaired window. If an operation is not assigned to m, its self-loop is active. If it is assigned to m, exactly one predecessor and one successor are chosen in the local circuit. In a simplified notation, the adjacency variables satisfy

$$\sum_{v \neq u} a_{u,v,m} = x_{u,m}, \qquad u \in \Omega_m, \tag{11}$$

$$\sum_{u \neq v} a_{u,v,m} = x_{v,m}, \qquad v \in \Omega_m. \tag{12}$$

The CP-SAT model uses a circuit constraint with self-loops for unassigned nodes and arcs from the depot to the first selected operation and from the last selected operation back to the depot.

Cleaning is represented on machine arcs. Let f_u be the route family of node u. For an arc (u, v, m), a family-switch cleaning is required if $f_u \neq f_v$. A count-triggered cleaning is required if the count after u has reached the threshold H_m. The mandatory cleaning duration is

$$\ell^{\text{man}}_{u,v,m} = \max\left\{\ell^{\text{sw}}_{u,v,m}, \ell^{\text{cnt}}_{u,v,m}\right\}, \tag{13}$$

where the two terms are zero unless their corresponding trigger is active. The temporal separation on an internal arc is

$$a_{u,v,m} = 1 \Rightarrow S_v \geq D_u + \ell^{\text{man}}_{u,v,m}. \tag{14}$$

The analogous head-link and tail-link constraints use $\bar{D}_{h_m}$ and $\bar{S}_{t_m}$.

The post-cleaning count state propagates along the selected arc. If a family-switch or count-triggered cleaning is inserted, the count after v is reset to one. Otherwise, the count increases by one:

$$a_{u,v,m} = 1, \quad r^{\text{sw}}_{u,v,m} + r^{\text{cnt}}_{u,v,m} \geq 1 \Rightarrow q^m_v = 1, \tag{15}$$

$$a_{u,v,m} = 1, \quad r^{\text{sw}}_{u,v,m} = r^{\text{cnt}}_{u,v,m} = 0 \Rightarrow q^m_v = q^m_u + 1. \tag{16}$$

The model enforces the threshold condition with an auxiliary Boolean b^m_o indicating whether $q^m_o = H_m$.

Idle-triggered cleaning is handled after the local sequence has been realized. For a realized adjacency (u, v, m), let $\Delta_{u,v,m} = S_v - D_u$ be the idle gap. When $\Delta_{u,v,m} > \Theta_m$, the validator requires a cleaning reservation of duration τ^{idl}_m to be present in the machine gap. Schedules without sufficient reserved gap are reported as machine-cleaning conflicts. This separation avoids treating idle-triggered cleaning as a binary decision inside every CP neighborhood.

4.4 Order constraints and objective

No-overtake constraints are imposed for operations with the same cassette, route, step, and selected machine. For each no-overtake pair (o, o') with ascending wafer order,

$$x_{o,m} = x_{o',m} = 1 \Rightarrow S_o \leq S_{o'}, \qquad m \in \mathcal{M}_o \cap \mathcal{M}_{o'}. \tag{17}$$

Output-order constraints are imposed on the final modeled operation of each job within a cassette-route group. Sequential-release instances also impose start-order constraints on first modeled operations across cassettes.

The local CP objective minimizes a compact score. The primary term is the compact makespan of the repaired schedule. Secondary terms include total release time and penalties for elastic suffix movement. The CP subproblem is solved under a short time limit. A repaired skeleton is therefore a local feasible or improved schedule for the selected neighborhood, not a certificate of global optimality.

Remark 1 (Reported feasibility)**.** *The CP repair model returns a compact machine-level skeleton. The final reported schedule is accepted only after decoding and validation. Thus reported schedules are executable with respect to the event-level checks used in this study. No global optimality guarantee is claimed for the full scheduling problem.*

5 Event-Level Decoding and Validation

A compact machine-level schedule is not sufficient for this benchmark because the required output is an executable MoveList. The decoder expands each accepted skeleton into actions. The validator then checks whether the realized event trace satisfies the equipment event rules and JIT timing conditions used in this study.

5.1 MoveList actions

The event representation contains the following action types: pick, place, transfer, prepare, complete, pump, vent, process, clean, and align. Pick, place, and transfer actions are associated with robot movement. Pump and vent actions are associated with load-lock state changes. Prepare and complete actions represent required equipment-side state transitions. Process, clean, and align actions are process-like module actions.

The decoder constructs a realized event trace from the compact schedule. The realized makespan is the maximum end time over non-clean executable events. Cleaning events are retained for feasibility and accounting, but the reported objective follows the benchmark convention.

5.2 Physical transition and handoff checks

The validator tracks the location of each wafer through the realized events. Initially, a wafer is in its load port. A pick action moves it onto a robot, a transfer action keeps it on the same robot while changing the

Table 5: Event-level validation checks.

Check	Meaning
PBA	Process-before-arrival conflict. A process-like action starts before the wafer has physically arrived at the destination module.
SWO	Same-wafer overlap conflict. Two realized actions for the same wafer overlap in time.
BH	Broken handoff conflict. The source, destination, or wafer-location state is inconsistent across consecutive actions.
IT	Impossible-transition conflict. A robot is asked to move between stations outside its domain.
JIT-anchor	Anchor inconsistency in the realized JIT calculation, usually caused by missing or inconsistent mandatory transition events.
JIT-dwell	The realized post-process dwell excess exceeds the 15-second dwell limit.
JIT-move	The realized move excess from one operation to the next exceeds the 30-second move limit.
Missing process	At least one modeled operation has no realized process-like event in the decoded trace.
Decoder early start	The decoder realizes a process-like event earlier than the compact skeleton permits.
Warning	Non-fatal mismatch between compact skeleton timing and realized event timing. Such warnings record unavoidable event-level delays and do not by themselves reject a schedule.

source and destination relation, and a place action moves it into the destination station. Prepare, pump, vent, complete, process, and align actions require the wafer to be located at the corresponding module and not on a robot.

The validator checks three classes of physical errors. First, it checks same-wafer overlap by ensuring that consecutive actions of the same wafer do not overlap in time. Second, it checks handoff continuity by verifying that each action starts from the location produced by the previous action. Third, it checks robot-domain feasibility. TM1 can serve the atmospheric-side stations, while TM2 and TM3 serve their respective vacuum-side station sets. A move outside the domain of the selected robot is an impossible transition.

5.3 Realized JIT timing

The compact CP model uses simple dwell and move guidance. The validator applies the exact JIT check on realized events. For a job arc (o, o'), let C_o^{real} be the realized completion time of the process-like action for o. Let D_o^{real} be the completion time of the first leave-side handoff after o, and let $A_{o'}^{\mathrm{real}}$ be the realized arrival or handoff completion time at o'. Let $\tau_o^{\mathrm{need,leave}}$ and $\tau_{o,o'}^{\mathrm{need,total}}$ be the mandatory transition times needed to leave the current node and to reach the next node. The validator checks

$$0 \le D_o^{\mathrm{real}} - C_o^{\mathrm{real}} - \tau_o^{\mathrm{need,leave}} \le 15, \tag{18}$$

$$0 \le A_{o'}^{\mathrm{real}} - C_o^{\mathrm{real}} - \tau_{o,o'}^{\mathrm{need,total}} \le 30. \tag{19}$$

The mandatory terms include actions such as pick, place, transfer, pump, vent, prepare, complete, valve opening, and valve closing. This subtraction is important because the benchmark limits the excess waiting beyond mandatory motion and equipment-state transitions, not the raw elapsed time between process events.

Table 6: Main experimental configuration.

Item	Value	Rationale
Global LNS time limit	20 s	Short contest-style computational budget used for the main comparison.
CP repair time limit	2 s	Bounded local repair time, which keeps each neighborhood iteration responsive.
Maximum LNS iterations	20	Iteration cap aligned with the short global time budget.
Parallel workers	2	Modest reproducible setting on the reported workstation.
Random seed	Seven seeds for main comparison; five seeds (11, 42, 47, 46, 2026) for ablation; three seeds (11, 42, 123) for sensitivity	Main table reports best across seven seeds per instance; ablation and short-horizon report five-seed means.
Neighborhood radius	4 for the default sensitivity setting	Local repair size used as the baseline in the reported time sweep.
Solver	C++17 with OR-Tools v9.10	CP-SAT solver stack used in the experiments.
Hardware	Ubuntu 22.04, Intel Core i7-11700 CPU, 32 GB RAM	Workstation used for the reported experiments.
Task scenarios	5a–5d base, 10a–10d adaptive	Scenario partition used by the benchmark data.

6 Computational Study

6.1 Experimental setup

The experiments use eight benchmark instances from the NAURA Cup setting[1]. Tasks 5a–5d use the base timing and cleaning configuration, while tasks 10a–10d use the adaptive configuration. The solver is written in C++17 and uses OR-Tools v9.10 for CP-SAT repair. Table **??** gives the main experimental settings.

The 20-second global time limit represents a short-response optimization setting rather than an exhaustive offline search. The 2-second CP repair limit prevents a single neighborhood from dominating the run time. The two-worker setting is intentionally modest, which makes the results easier to reproduce on a workstation. These parameter values are not treated as globally tuned constants. The sensitivity analysis below shows that the neighborhood radius can matter more than simply increasing the global time limit.

6.2 Main comparison

Table **??** compares the default initial construction, the best LNS result, and the reported results of Liu et al. [**?**]. The LNS-best column reports the best result across seven seeds for each instance. All reported schedules are accepted by the implemented event-level validator. The LNS stage improves the initial schedule

[1]Official competition website: `http://univ.ciciec.com/nd.jsp?id=876#_jcp=1`.

Table 7: Comparison of the initial heuristic, the best LNS result, and Liu et al. [**?**]. Gap is computed as (LNS − Liu)/Liu.

Instance	Initial heuristic		LNS best		Liu et al.		Gap
	Obj.	Time/s	Obj.	Time/s	Obj.	Time/s	
5a	44107.5	0.09	44107.5	7.87	51861.5	200.00	-14.95%
5b	12258.5	0.03	12200.5	11.51	12650.0	22.30	-3.55%
5c	8806.5	0.04	8806.5	17.84	10562.5	7.70	-16.62%
5d	13507.5	0.03	13507.5	19.51	12701.5	7.60	+6.35%
10a	18456.5	0.04	18384.5	12.94	22646.0	36.80	-18.82%
10b	26306.5	0.04	26165.5	17.88	27518.5	111.60	-4.92%
10c	15468.5	0.03	15402.5	11.50	15736.0	7.50	-2.12%
10d	24238.5	0.06	24208.5	21.36	19061.5	200.00	+27.00%

on 5b, 10a, 10b, 10c, and 10d under the main-run configuration. It does not improve the initial schedule on 5a, 5c, and 5d within the same budget.

The method improves the reported objective of Liu et al. [**?**] on six of the eight instances. The largest relative gains occur on 5a, 5c, and 10a. The method gives worse objective values on 5d and 10d. The runtime values of Liu et al. are taken from their reported results and are included only for context, since the computational environments and implementations are not identical.

Liu et al. [**?**] is the main benchmark because it is the most relevant publicly reported system for the same competition family. It uses the same task naming convention, targets executable scheduling outputs, and was reported as a strong competition solution.

6.3 Analysis of non-dominating instances

The two non-dominating instances show different failure modes. On 5d, the initial construction already gives the final result found by LNS under the tested budgets, and neither longer time limits nor larger radii in the reported sensitivity data produce improvement. This suggests that the current neighborhoods do not expose a useful improving move for this instance, or that improvement requires a structural change not reached by the implemented destroy operators.

Instance 10d behaves differently. The main run gives a worse objective than Liu et al., but the radius sweep shows that smaller neighborhoods can produce substantially better results than the default radius on some seeds. This indicates that 10d is sensitive to neighborhood design. The result does not undermine the use of CP repair, but it shows that selecting the right relaxed region is at least as important as optimizing the selected region. This observation motivates stronger bottleneck-aware neighborhoods and offline initialization of operator or radius choices in future work.

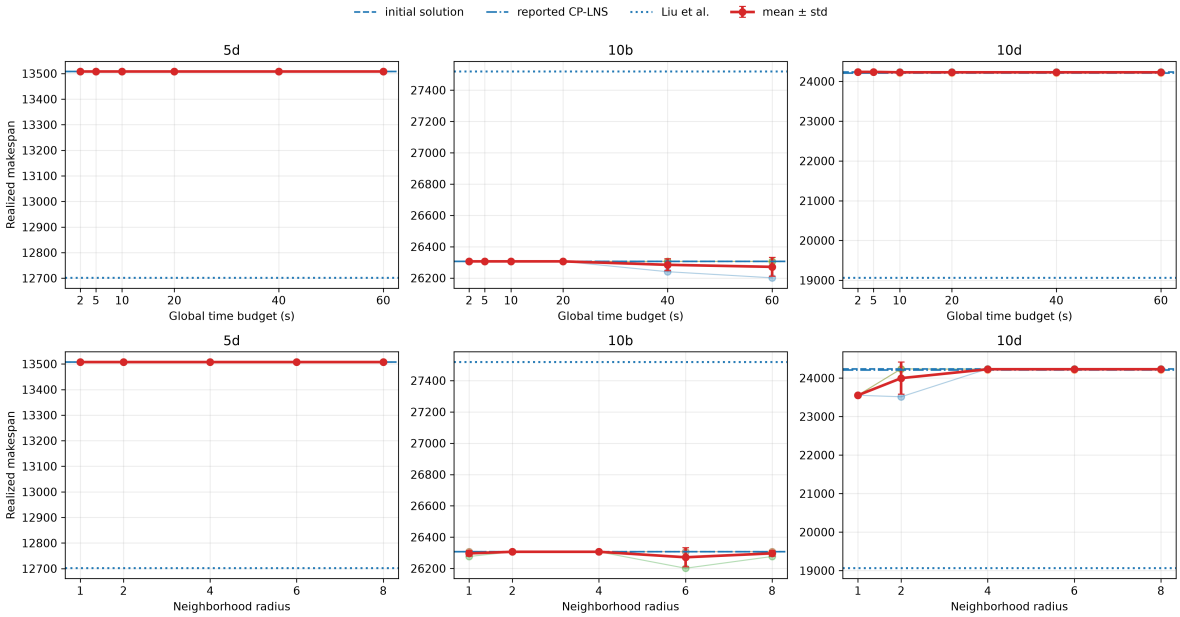

Figure 2: Sensitivity of realized objective values to global time limit and neighborhood radius on representative instances.

6.4 Sensitivity and short-horizon performance

Figure **??** summarizes the effect of the global time limit and neighborhood radius on representative instances. On the tested instances, solution quality is more sensitive to the neighborhood radius than to time alone. This is consistent with the decomposition of roles in the algorithm, where LNS decides which part of the schedule is relaxed and CP repairs that part locally.

Table **??** reports a stricter short-horizon sweep over all eight instances and five seeds. The sweep uses the default initial construction and the adaptive CP-LNS policy with 1-second, 5-second, and 20-second global budgets. The CP repair budget is truncated by the remaining global time, so a short run is not extended by a single long repair. All 120 runs are validator-feasible and have zero event-level process-before-arrival, same-wafer-overlap, broken-handoff, impossible-transition, JIT-anchor, JIT-dwell, and JIT-move conflicts. The mean objective improves from 20392.300 at 1 second to 20378.225 at 20 seconds. Most of the 20-second targets are already reached earlier, where 32 of 40 runs match their 20-second objective at 1 second and 36 of 40 do so at 5 seconds.

6.5 Ablation evidence and ablation protocol

We report a component ablation that starts from the current default initial construction and compares the initial solution alone, static CP-LNS, and adaptive CP-LNS under the same validator and budget. The ablation uses a 20-second global limit, a 2-second CP repair limit truncated by the remaining global time, a 50-iteration cap, and two CP workers, with seeds 11, 42, 47, 46, and 2026. Every accepted sched-

Table 8: Short-horizon time sweep over all eight instances and five seeds. The final-target column counts runs whose objective matches the corresponding 20-second result for the same instance and seed.

Budget/s	Runs	Feasible	Mean obj.	Improvement	Final target	Mean time/s
1	40	40	20392.300	0.007%	32/40	1.027
5	40	40	20385.000	0.043%	36/40	5.036
20	40	40	20378.225	0.076%	40/40	20.022

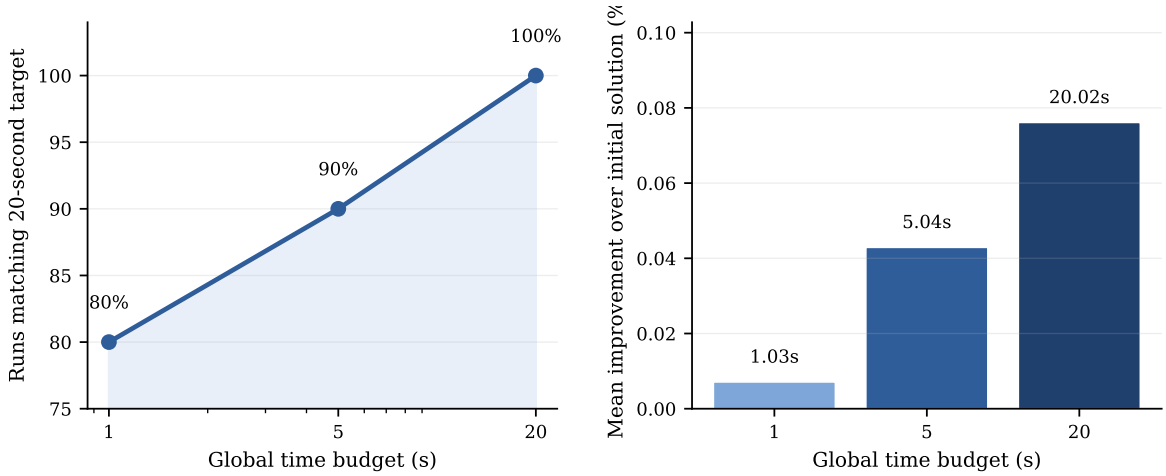

Figure 3: Short-horizon target attainment and mean improvement over the default initial construction. The left panel reports the share of runs that match the corresponding 20-second objective for the same instance and seed.

ule is validator-feasible and has zero event-level process-before-arrival, same-wafer-overlap, broken-handoff, impossible-transition, JIT-anchor, JIT-dwell, and JIT-move conflicts.

The ablation separates CP repair from adaptive neighborhood selection. We compare the default initial construction alone, the default initial construction followed by CP-LNS with static operator weights and no conflict-focused neighborhoods, and the default initial construction followed by the adaptive CP-LNS policy. The static policy keeps fixed destroy-operator weights and therefore removes the online reward update. The adaptive policy uses the implemented reward-based operator update with conflict-focus probability 0.65. Because all initial incumbents are feasible in these runs, the conflict-focused neighborhood is not activated.

Table **??** gives three conclusions. First, the initial construction accounts for most of the objective variation across instances. Second, CP repair preserves feasibility and performs nontrivial local search, but its objective contribution is small under the tested 20-second budget. Starting from the default initial construction, static CP-LNS improves the mean objective by 0.090%, and the adaptive CP-LNS variant improves it by 0.076%. Third, the data-guided adaptive policy changes search behavior but does not improve the final objective in this feasible-start setting. Figure **??** compares adaptive and static CP-LNS on matched instance–seed pairs. The horizontal value is $C_{\max}^{\text{adaptive}} - C_{\max}^{\text{static}}$, so points to the left of zero favor adaptive CP-LNS and points to

Table 9: Component ablation over all eight instances and five seeds. The improvement column is measured relative to the default initial construction alone.

Configuration	Runs	Mean obj.	Sum obj.	Improvement	Time/s	Iter.	Feas. repairs	Accepted	Best updates
Default initial construction	40	20393.750	815750.0	0.000%	0.075	0.000	0.000	0.000	0.000
Default + static CP-LNS	40	20375.425	815017.0	0.090%	20.025	22.925	9.500	7.825	0.350
Default + adaptive CP-LNS	40	20378.225	815129.0	0.076%	20.022	23.525	10.025	8.525	0.275

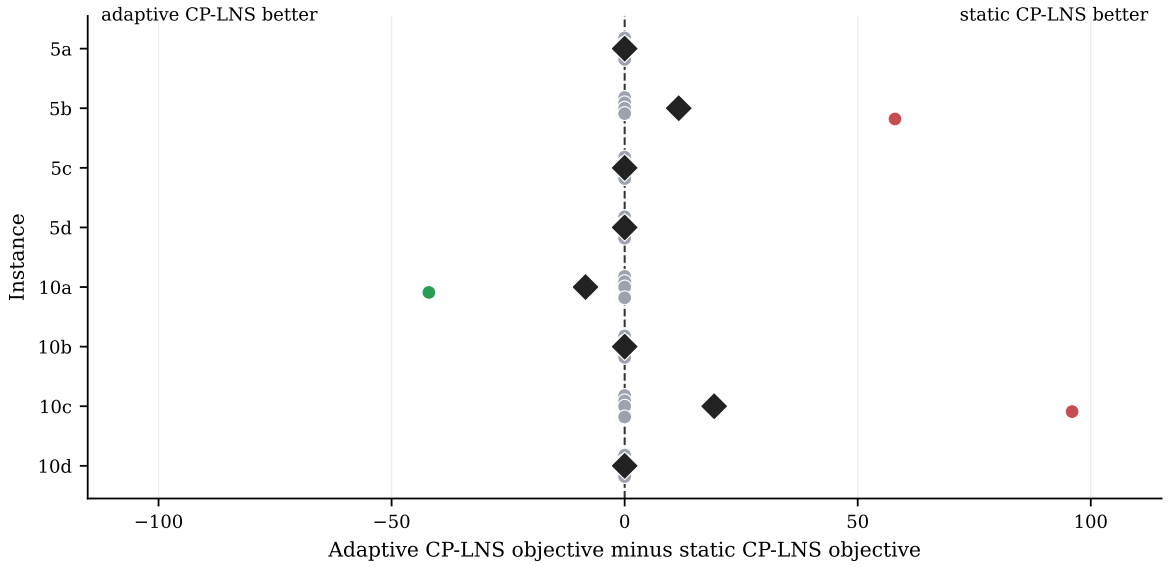

Figure 4: Paired objective difference between adaptive and static CP-LNS. Each dot is one matched instance–seed pair, and each diamond is the instance mean. The plotted value is $C_{\max}^{\text{adaptive}} - C_{\max}^{\text{static}}$, where negative values favor adaptive CP-LNS and positive values favor static CP-LNS.

the right favor static CP-LNS. The adaptive variant has more feasible repairs and accepted moves than the static variant, but the static variant has a lower mean objective and more best updates.

We also ran a JIT stress test for a failure mode in which an otherwise feasible initial schedule is perturbed by extending one blocking release and thereby creating one JIT dwell and one JIT move violation. This test is not part of the main comparison, because it starts from an artificially damaged schedule. Table **??** shows that conflict-focused sampling can identify the JIT conflict, but it does not improve recovery in this setting. With seed 42 over the eight instances, the static policy without conflict focus recovers feasibility in five cases, whereas the conflict-focus probability 0.65 setting recovers two cases. This result supports a conservative claim. The reported main results are feasible under the event-level validator, but the current CP repair should not be presented as a general repair method for arbitrary JIT damage.

The sensitivity of 10d to neighborhood design motivates a further test of whether using only the coordinated time-window neighborhood improves the results. The time-window-only policy assigns all ordinary destroy-operator probability to the time-window operator and disables conflict-focused neighborhoods. Table **??**